
CampusMonk
By Rachit Rastogi

CampusMonk
By Rachit Rastogi

HASHING

C++ Java Python

Map/HashMap/Dictionary

Map

Key	Values
"Anshul"	1285
"XyZ"	936
{String	int}

Key	Value
char	integer
String	-
integer	-

CampusMonk
By Rachit Rastogi

Ques:

Given a string find the number of permutations possible from it's characters.

Permutation - A Permutation of a string is another string that contains same characters, only the order of characters can be different.

Example:

Input 1: abc

Output 1: 6

Input 2: abcc

Output 2: 12

Input 3: hello

Output 3: 60

Solution:

C++

```
#include<bits/stdc++.h>
using namespace std;
int fact(int n)
{
    if(n==0||n==1) return 1;
    return n*fact(n-1);
}int main()
{
    unordered_map<char, int> mp;
    string s;   cin>>s;
    for(int i=0;i<s.size();i++)
    {
        mp[s[i]]++;
    }
    int permutation = fact(s.size());
    for(auto x: mp)
    {
        permutation/=fact(x.second);
    }
    cout<<permutation;
}
```

Java

```
import java.util.*;
class HelloWorld {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        HashMap<Character, Integer> mp = new
HashMap<>();
        for(int i=0;i<s.length();i++)
        {
            int count = mp.containsKey(s.charAt(i)) ?
mp.get(s.charAt(i)) : 0;
            mp.put(s.charAt(i), count + 1);
            }
            int permutation = fact(s.length());
            for (int freq : mp.values())
        {
            permutation/=fact(freq);
        }
        System.out.println(permutation);
    }
    public static int fact(int n)
    {
        if(n == 0 || n == 1) return 1;
        return n * fact(n-1);
    }
}
```

Python

CampusMonk
By Rachit Rastogi

```
def fact(n):
    if(n==0 or n==1):
        return 1
    return n*fact(n-1)
s = input()
mp = {}
for i in range(len(s)):
    mp[s[i]] = mp.get(s[i],0) + 1
permutation = fact(len(s))
for freq in mp.values():
    permutation = permutation // fact(freq)
print(permutation)
```

Ques:

Nitin is assigned a sequence, but he has to convert this sequence into a very custom sequence. This requirement has been setup by his client. So the end sequence should look something like below

CampusMonk
By Rachit Rastogi

Suppose the sequence is A (A1, A2, A3,...AN) of N elements.

There exists a value X such that 1 <= X <= N, such that:

1. The sequence should be strictly increasing from index 1 to X.

2. The sequence should be strictly decreasing from index X to N. You can re-arrange the elements if required.

We describe the problem in simpler terms as, create a new sequence such that first X elements are strictly increasing, and the remaining elements are strictly decreasing Use the elements from A[] only once.

Find that value of x, for which the above condition satisfies

For eg we have sequence A[] = [1, 9, 8, 3, 4, 5] Then from this sequence we can make the sequence as completely increasing as [1, 3, 4, 5, 8, 9]

[1, 9, 8, 3, 4, 5] - True

1,2,2,3 – True(1,2,3,2)

1,3,3,2 - False

Solution:

C++

```
#include<bits/stdc++.h>
using namespace std;
int main()
{
    unordered_map<int, int>mp;
    int n;  cin>>n;
    int arr[n];
    int mx = 0;
    for(int i=0;i<n;i++)
    {
        cin>>arr[i];
        mx = max(mx, arr[i]);
        mp[arr[i]]++;
    }
    for(auto x: mp)
    {
        if((x.first == mx && x.second > 1) || x.second > 2)
        {
            cout<<"False";
            return 0;
        }
    }
    cout<<"True";
}
```

JAVA

```
import java.util.*;
class HelloWorld {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        HashMap<Integer, Integer> mp = new
 HashMap<>();
        int[] arr = new int[n];
        int mx = 0;
        for(int i=0;i<n;i++)
        {
            arr[i] = sc.nextInt();
            if(arr[i]>mx){
                mx = arr[i];
            }
            int count = mp.containsKey(arr[i]) ?
 mp.get(arr[i]) : 0;
            mp.put(arr[i], count + 1);
        }
        for (int key : mp.keySet())
        {
            if((key == mx && mp.get(key) > 1) ||
 mp.get(key) > 2)
            {
                System.out.println("False");
                System.exit(0);
            }
        }
        System.out.println("True");
    }
}
```

Python

```
# take input from user
arr = [1, 9, 8, 3, 4, 5]
mp = {}
mx = 0
for i in range(len(arr)):
    mp[arr[i]] = mp.get(arr[i],0) + 1
    if(arr[i]>mx):
        mx = arr[i]
for key in mp.keys():
    if((key == mx and mp[key] > 1) or mp[key] > 2):
        print("False")
        exit(0)
print("True")
```

CampusMonk

By Rachit Rastogi



Download our APP :-

GET IT ON Google Play

Download on the App Store

(Link in Description)

Ques

Recently Aarush has learnt Advance Mathematics. He was exploring multi-digit numbers. Aarush was trying finding divisors of different numbers. He came across one scenario where he is having different numbers and need to check if sum of proper divisor of each number is equal to the other number. Write a program to check if sum of proper divisor of first number N will equal to the other number M and vice versa. Print "Yes" if equal, otherwise "No". and if out of range print "Wrong Input".

Range of Input is 1 <= N,M <= 100000

Example 1: Input:

220 -> Integer number N

284 -> Integer number M.

Output:

Yes -> result-String

Explanation:

Give two numbers 220 and 284. Divisors of 220 are 1, 2, 4, 5,10, 11, 20, 22, 44, 55 and 110. Sum of these is 284 (Equal to Second Number). To check vice versa scenario, divisors of 284 are 1, 2, 4, 71 and 142 with sum 220. Hence output will be Yes.

Example 2:

Integer number N = 2

Integer number M = 4

Output:

No → result - String

Explanation:

Give two numbers 2 and 4. A divisor of 2 is only 1. Sum is 1 (Not equal to Second Number). Hence output will be No

HomeWork

1. find character/Integer with maximum frequency in given string/Array
2. find character/Integer with minimum frequency in given string/Array
3. check if two strings are anagram of each other or not

